CobaltC Programming Language Specification
Version 1.0.0 — Publication Edition

OWNERSHIP • LIFETIMES • SAFETY • CONTROL

CobaltC Programming Language Specification

OWNERSHIP • LIFETIMES • SAFETY • CONTROL

CobaltC 1.0 — Specification version 1.0.1

Publication Status: This edition defines the CobaltC 1.0 language contract. Implementation technique, representation, and compiler architecture remain implementation choices except where explicitly constrained by normative language guarantees.

NOTE: Within the narrative and theoretical scope of the book , Appendix 06 serves primarily to demonstrate how a memory-safe, C-successor language ought to define its semantic boundaries, rather than to provide a drop-in replacement for a rigorous ISO-level standard.

Publication Edition: The formatted PDF edition of this specification is available here .

Preface

As argued throughout The Wrong Memory essays, attempting to tack a "safe subset" onto an inherently unsafe language usually fails because the core language semantics still allow invariant violations. By embedding ownership and borrowing into the core type system, CobaltC makes safety an intrinsic property of the language rather than an optional discipline imposed upon it.

CobaltC unashamedly draws on the ownership and memory-safety techniques demonstrated by Rust. Although Rust isn't the origin of most of these ideas, its significant contribution was demonstrating how they could be combined into a practical, high-performance systems language. CobaltC adopts that safety architecture without attempting to reproduce Rust's syntax or overall language philosophy. Instead, it applies the broader body of ideas to the problem identified in Appendix II: making semantic invariants such as ownership, lifetime, aliasing, mutability, and bounds properties that the compiler can represent and enforce, rather than obligations the programmer must maintain by convention.

Contents

1. Introduction

This specification defines what a conforming implementation MUST do; it does not prescribe a particular implementation technique unless this document explicitly makes an implementation property normative.

CobaltC is a statically typed systems programming language providing explicit ownership, deterministic destruction, compiler-checked borrowing, inferred lifetimes, explicit nullability, bounds-safe operations, structured error handling, safe concurrency, explicit unsafe operations, and explicit foreign-function interfaces.

The language is intended for software requiring predictable resource management, strong memory safety, native execution, and controlled interaction with low-level facilities.

Core Design:

CobaltC is C-like data plus ordinary functions, generics, explicit memory and ownership safety, and modules.

Managed pointers are the source-language representation of borrow capabilities. A borrow is the semantic relationship between an access capability and a referent; a managed pointer is the value that carries that capability. Managed pointers do not represent ownership unless a language rule explicitly states otherwise.

CobaltC provides C-like systems programming, but ownership, lifetime, bounds, nullability, and data-race rules are language semantics rather than programmer convention.

CobaltC provides deterministic resource management through destruction at language-defined ownership and scope boundaries and compiler-enforced borrowing rather than relying on garbage collection or programmer convention.

Visibility is defined at the module boundary through explicit export declarations; CobaltC does not use public , private , or protected declarations.

Unsafe operations and foreign interfaces form explicit boundaries outside the automatic safety guarantees of safe CobaltC.

CobaltC does not provide classes, object-oriented inheritance or dynamic dispatch.

The specification deliberately leaves implementation technique, representation, and compiler architecture to the implementor except where those choices affect a normative language guarantee. Examples in this specification illustrate normative syntax and semantics unless explicitly stated otherwise. Examples do not introduce additional language features or requirements beyond those defined by the normative text.

2. Normative Terminology

The words MUST , MUST NOT , SHOULD , SHOULD NOT , and MAY are normative.

CobaltC 1.0 refers to the language edition. Specification version 1.0.1 refers to this document revision.

Implementation-defined means that an implementation chooses the behavior from the alternatives permitted by this specification and documents that choice.

Unspecified means that this specification permits more than one behavior and does not require an implementation to document which permitted behavior it chooses.

Undefined behavior is behavior for which this specification imposes no requirements. Operations expressible in safe CobaltC without entering an unsafe context MUST NOT have undefined behavior.

Managed pointer value refers to a source-language value that carries a managed access capability. The capability itself is the borrow relationship; the managed pointer value is the representation of that capability in source code.

Owned value refers to a value whose lifetime and destruction are controlled by the owning storage location according to the ownership rules of this specification. A borrowed managed pointer is not an owned value unless a language rule explicitly states otherwise.

Unsafe context is a program region in which operations outside the guarantees of safe CobaltC are permitted. An unsafe operation is an individual operation requiring such a context. Unsafe code is code containing or invoking unsafe operations.

3. Source Files

A CobaltC program consists of one or more source modules.

A translation unit defines exactly one module. The module declaration of a translation unit establishes the module defined by that translation unit. A translation unit MUST contain exactly one module declaration.

Submodules are distinct modules and MUST be defined by separate translation units. A submodule is not part of its parent module merely because it is organized beneath that module in a source-file hierarchy.

An implementation MAY use any source-file, directory, project, or compilation-unit organization internally, provided that the resulting module structure and behavior conform to this specification.

Source text is UTF-8 encoded Unicode. The source representation is interpreted as Unicode scalar values. Unicode normalization is not applied to identifiers; identifier comparison uses the source code-point sequence after lexical classification.

Identifiers are case-sensitive.

Whitespace separates lexical tokens where necessary and otherwise has no semantic meaning. An implementation MAY accept additional source encodings as extensions, but a conforming implementation MUST accept UTF-8.

Module Visibility

Module Visibility: CobaltC deliberately does not require public , private , protected , or equivalent visibility attributes on individual declarations.

Visibility is expressed at the module boundary. Declarations describe the implementation of a module, while the module's export block describes the declarations made available as part of its external interface. In this specification, exported describes declarations intentionally made available by a module, accessible describes whether code may legally reference a declaration, and visible is reserved for ordinary lexical or name-resolution concepts. This terminology distinction applies throughout this specification.

A declaration is externally accessible only when it is exported by its defining module. Nothing is exported implicitly.

A declaration that is not exported remains available only within the access permitted by the ordinary name-resolution and module rules. External code MUST NOT access a non-exported declaration merely because it knows its name or because the declaration appears in a source file belonging to the module.

The export boundary provides an explicit software-engineering boundary: declarations that are not exported MAY be changed without changing the externally visible interface, provided that the specified behavior and requirements of the exported interface remain compatible.

The same principle applies to submodules. A module hierarchy provides organization and namespaces but does not weaken encapsulation. Each module independently controls its external contract, and a parent module does not implicitly export, import, or expose declarations from a submodule.

An implementation MAY provide additional visibility mechanisms as extensions, but such mechanisms MUST be distinguishable from standard CobaltC behavior and MUST NOT silently alter the semantics of a valid CobaltC 1.0.1 program.

Source-File Organization

The language does not prescribe a particular source-file or directory layout. For example, a real CobaltC program might be organized as follows:

```
src/
```

```
    main.cb
    cli.cb

    database/
        database.cb
        record.cb
        index.cb

    filesystem/
        scanner.cb
        path.cb

    text/
        tokenizer.cb
        matcher.cb
```

This layout is illustrative only. The language defines modules and their relationships through module declarations, imports, and exports, not through directory names or file-system conventions.

4. Comments

CobaltC supports line comments:

```
// comment
```

A line comment extends to the next line terminator or the end of the source file.

CobaltC also supports block comments:

```
/*
   comment
*/
```

Block comments may be nested:

```
/* outer comment
   /* nested comment */
   still inside outer comment
*/
```

Nested block comments are useful for temporarily commenting out blocks of code that already contain block comments.

When lexing a block comment, the nesting depth is initially one. Each /* encountered within the comment increases the nesting depth by one, and each */ decreases it by one. The block comment ends when the nesting depth reaches zero.

An unterminated block comment is a lexical error. Comments have no semantic effect.

5. Keywords

The following words are reserved:

```
alias
as
break
const
continue
defer
else
enum
export
extern
false
fn
for
if
import
in
loop
match
```

```
move
mut
null
return
struct
true
unsafe
while
```

drop is reserved for compiler-recognized destruction semantics but has no ordinary source-language production.

alias introduces a type-alias declaration.

An alias declaration has the form alias Name = ExistingType; . An alias is transparent for type compatibility and does not create a new nominal type.

The word let is not a CobaltC keyword and is not introduced by this edition.

A keyword is recognized only when the complete lexical token is the keyword; a longer identifier such as returnValue remains an identifier.

6. Identifiers

An identifier begins with a Unicode identifier-start character and may contain subsequent identifier characters and digits. Identifiers are case-sensitive.

```
value
Value
VALUE
```

Identifier Security and Unicode Restrictions

User-defined identifiers MUST comply with Unicode Technical Standard #39 (TR39) under the Moderately Restrictive security profile. Format characters, zero-width spaces such as U+200B, and bidirectional text override characters such as U+202E MUST NOT appear in an identifier token.

The implementation MUST diagnose an identifier when the applicable TR39 confusables data establishes a prohibited conflict with a protected keyword, predefined core type, or conflicting identifier in the applicable lookup scope.

7. Literals

CobaltC provides integer, floating-point, character, string, Boolean, and null literals.

Integer literals

Integer literals may be written in decimal, hexadecimal (0x), binary (0b), or octal (0o) form. An underscore may separate digits within a literal but MUST NOT appear at the beginning, end, or twice consecutively.

Floating-point literals

A floating-point literal contains a decimal point and/or an exponent. The default floating-point type is f64 when no contextual type is available.

Character literals

A character literal is enclosed in single quotes and denotes exactly one Unicode scalar value. Standard escapes include \n , \r , \t , \0 , \\ , and \' .

String literals

A string literal is enclosed in double quotes and produces a String value. Standard escapes include \n , \r , \t , \0 , \\ , and \" . A string literal MUST denote valid UTF-8.

Literal typing

Integer literals are typed from context where possible; without a contextual integer type, the implementation uses i32 when the value is representable and MUST otherwise require an explicit type. A literal whose value cannot be represented by its required type is a compile-time error.

8. Modules

A module declaration has the form:

```
module example;
```

A module establishes a namespace. A translation unit MUST contain exactly one module declaration.

Modules MAY import declarations, or selective declarations, from other modules:

```
import io;
import io { print, println };
```

Name resolution is lexical and module-aware. An unresolved name is a compile-time error. Imports do not implicitly re-export declarations.

Module Contracts

A module's export { ... } block lists declarations externally accessible from that module. Declarations not listed remain module-local. A submodule is a distinct module and is not exported by its parent merely because it exists beneath that parent.

A module hierarchy does not implicitly grant access between parent, child, or sibling modules. External code imports the module whose exported declarations it intends to use.

Two translation units MUST NOT define the same module for one program. Circular module dependencies MAY be supported, but name resolution MUST remain well-defined; an unresolved dependency is a compile-time error.

9. Declarations

CobaltC provides:

```
const
alias
struct
enum
fn
```

Declarations are introduced into their applicable lexical or module namespace.

Inner declarations MAY shadow outer declarations where permitted.

10. Variables

A variable is declared using:

```
i32 count = 0;
```

A mutable variable is declared using:

```
mut i32 count = 0;
```

An uninitialized declaration is permitted:

```
i32 result;
```

but result MUST be initialized before it is read. A declaration's initializer is evaluated after the binding is introduced but before the binding is considered initialized; referring to the binding during its own initializer is therefore invalid.

11. Constants

Constants use:

```
const i32 maximum = 100;
```

A constant initializer MUST satisfy the implementation's constant-expression requirements, and an implementation MUST document those requirements.

A constant cannot be mutated or borrowed mutably.

12. Primitive Types

CobaltC defines:

```
bool
char

void

i8
i16
i32
i64
i128

u8
u16
u32
u64
u128

isize
usize

f32
f64
```

The fixed-width integer types have exactly their specified widths. Signed integers use two's-complement mathematical ranges. isize and usize are pointer-sized signed and unsigned integer types respectively.

char represents one Unicode scalar value. bool has exactly two values, true and false . void is the return type of functions that produce no value.

Integer overflow, division by zero, and invalid shift operations are not silently memory-unsafe. An implementation MUST diagnose statically provable invalid constant operations and MUST otherwise apply the failure semantics specified by Section 27.

The implementation provides predefined integer-limit constants such as usize_max , whose values correspond to the greatest representable value of the named type. These constants are language-provided values and do not imply a particular representation or compiler implementation strategy.

13. Compound Types

CobaltC supports structs, enums, arrays, function types, managed pointers, raw pointers, and generic types. Types such as String , Vector , and Result described later in this specification are specified language-provided abstractions rather than primitive types.

Structs and enums are nominal types. Type aliases do not create new nominal types.

Managed pointer forms are:

```
T*
mut T*
T*?
mut T*?
```

Array types use T[N] . Function types use the function declaration form described by Section 24.

14. Managed Pointer Types

A managed pointer type specifies the referenced type, whether the pointer is nullable, and the access capability held by the pointer. A managed pointer is a borrow capability rather than an ownership capability unless a language rule explicitly states otherwise.

A pointer type is formed by appending * to the pointee type. For example, String* denotes a pointer to String .

A shared managed pointer permits read-only access. A mutable managed pointer permits exclusive mutable access for the duration of the applicable borrow.

Taking a shared borrow produces T* ; taking a mutable borrow produces mut T* :

The following forms illustrate the two borrow operations:

```
String* p = &value; // taking a shared borrow
mut String* p = &mut value; // taking a mutable borrow
```

A mutable managed pointer may be reborrowed or converted to a shared managed pointer. The shared reborrow temporarily prevents conflicting use of the original mutable capability while it remains live.

Shared managed pointers are copyable capabilities. Mutable managed pointers are moveable but are not implicitly copyable. Copying a shared managed pointer creates another shared borrow of the same referent; it does not create ownership.

Managed pointers are non-owning references unless a language rule explicitly states otherwise. Destroying or discarding one does not destroy its referent.

15. Raw Pointers

Raw pointers are represented by:

```
raw T*
```

A raw pointer is an unmanaged address referring to a value of type T . Raw pointers do not participate in the ordinary managed ownership, borrowing, lifetime, or destruction guarantees of CobaltC.

A raw pointer value MAY represent an address that is null, invalid, dangling, or otherwise refer to memory that is not currently a valid value of type T . The language does not automatically establish the validity or lifetime of a raw pointer.

Raw-pointer dereference, conversion between raw pointers and managed pointers, pointer arithmetic, and unrestricted manipulation of raw addresses require an unsafe context unless another language rule explicitly permits the operation.

A raw pointer MUST NOT be used to bypass the ownership or borrowing rules of safe CobaltC code. Code that uses raw pointers to access managed memory is responsible for maintaining the validity, lifetime, alignment, and aliasing requirements of the accessed value.

The compiler MUST NOT infer ownership, borrowing, or lifetime guarantees from the existence of a raw pointer.

An unsafe context is a source region in which unsafe operations are permitted. An unsafe operation is an operation whose correctness is not established by the ordinary guarantees of safe CobaltC. Unsafe code is code containing one or more unsafe contexts or unsafe operations. Entering an unsafe context does not suspend unrelated language guarantees; code remains subject to all applicable rules unless an explicitly unsafe operation permits otherwise.

16. Mutability

The mut qualifier may appear only once in a single binding declaration or type qualification. Repeated application of mut is a compile-time error.

Mutability is a semantic property that determines whether a value may be modified through a particular binding or managed pointer capability. When applied before a declaration, `mut` modifies the mutability of the binding. When applied before a type, `mut` modifies the access capability of that type. Qualifiers immediately preceding a type modify the type; qualifiers immediately preceding a declaration modify the binding.

The mut qualifier may appear only once in a single type or declaration position. Repeated or conflicting mutability qualifiers are invalid.

A mutable binding permits mutation of the value associated with that binding where no ownership or borrowing rule prohibits the operation.

A shared managed pointer of type T* provides read-only access to its referent. A mutable managed pointer of type mut T* provides exclusive mutable access to its referent for the duration of the live mutable borrow.

Binding mutability and managed-pointer mutability are distinct semantic properties. A mutable binding does not by itself create a mutable managed pointer.

For function parameters whose type is a managed pointer, the mutability specified by the managed pointer type determines the access capability of the parameter. A separate binding-level mut is not required to obtain mutable access through a mut T* parameter.

Thus:

```
fn update(mut String* value) { append(*value, "!"); }
```

declares a parameter of type mut String* . The parameter provides exclusive mutable access to its referent subject to the ordinary borrowing and lifetime rules.

Mutability does not override ownership, aliasing, borrowing, or lifetime rules.

The compiler MUST reject any operation that would mutate a value through a shared managed pointer or otherwise violate the exclusivity requirements of a mutable managed pointer.

17. Type Compatibility

Assignments, function arguments, and return values MUST have compatible types.

For managed pointers, compatibility accounts for referenced type, access capability, and nullability. T* and mut T* are distinct types, and a shared pointer cannot be used where mutable access is required.

A mutable managed pointer MAY be used where a shared managed pointer is required, subject to borrowing and lifetime rules. This represents a shared reborrow and temporarily suspends conflicting mutable use.

Implicit conversions MUST NOT silently remove nullability, create ownership, destroy ownership, increase access capability, invalidate a lifetime guarantee, or perform unsafe reinterpretation.

An explicit conversion MAY be provided, but an explicit conversion MUST NOT be a mechanism for bypassing the ownership, borrowing, lifetime, or safety rules of safe CobaltC.

18. Type Inference

CobaltC permits type inference where the grammar and surrounding context establish a unique type.

Inference MUST preserve ownership, binding mutability, managed-pointer access capability, nullability, borrowing, and lifetime distinctions.

A shared borrow infers T* ; a mutable borrow infers mut T* . Inference MUST NOT infer mutable access from shared access alone.

When no unique type can be established, the implementation MUST require an explicit type rather than choose an arbitrary type.

The compiler MAY implement inference using regions, constraints, control-flow analysis, graphs, or another sound technique. Those techniques are not additional source syntax.

19. Generic Types and Functions

Generic types and functions are statically checked. A generic declaration defines one parameterized declaration; supplying type arguments specializes that declaration. The implementation technique used for specialization is not constrained by this specification provided that observable language semantics are preserved.

```
fn identity<T>(T value) : T
{
    return value;
}
```

Generic type arguments MUST satisfy the ordinary type-compatibility and ownership rules applicable to the instantiated declaration. A generic declaration MUST NOT rely on properties of a type argument that are not established by the declaration's constraints or by the ordinary rules of the language.

CobaltC 1.0 does not define a user-facing generic type-constraint syntax. Generic declarations may therefore rely only on properties guaranteed for all possible type arguments and operations explicitly available under the ordinary rules of the language. CobaltC 1.0 intentionally provides unconstrained generics only; user-facing generic constraints are outside this edition. This limitation is a deliberate edition boundary rather than an omission of implementation capability. An operation that requires additional properties is valid only when those properties are universally available or established by the ordinary rules of the instantiated type.

20. Exported Types

Exporting a type makes the type name available to external code but MUST NOT, by itself, make the representation of that type available to external code.

External code MAY name an exported type, use the type in function signatures, and pass or return values or managed pointers to the type, subject to the ordinary rules of the type system.

External code MUST NOT directly inspect, access, or manipulate the representation of an exported type unless that representation is explicitly exposed by a language mechanism defined by this specification. In particular, external code MUST NOT access fields, layout, or other representation details solely because the type is exported.

The defining module controls how an exported type is constructed, accessed, and manipulated through its exported declarations.

An exported type MAY therefore be used as an opaque type by external code while remaining fully defined within its defining module.

21. Structs

A struct defines a nominal aggregate:

```
struct Point
{
    i32 x;
    i32 y;
}
```

Struct fields have declared types. A struct value owns its owned fields according to their declared types and the language's ownership rules. A field is owned when its declared type represents an owned value rather than a borrowed capability. Destruction applies only to owned values; borrowed managed pointers do not participate in destruction of their referents.

Struct construction uses the form Type { field = expression, ... } . Field names identify the destination fields, and each field MUST be initialized exactly once.

```
Point point = Point
{
    x = 10,
    y = 20
};
```

Structs do not contain function declarations. Operations on a struct may instead be declared as associated functions using the type's qualified name.

Destruction

A type MAY define a compiler-recognized destruction hook using associated-function syntax of the form fn T::destroy(mut T* value) . A destruction hook uses associated-function syntax but is a distinct language mechanism: it is invoked by compiler-defined destruction semantics and is not an ordinary associated function. It MUST NOT be invoked explicitly as part of normal source-language execution.

The existence of a destruction hook does not otherwise define method syntax, receiver semantics, or object-oriented behavior.

The hook has privileged exclusive access to the owned value while destruction is performed and releases resources directly managed by T that are not represented by owned fields. The hook MUST NOT independently destroy owned fields that the compiler will subsequently destroy. The compiler recursively destroys owned fields after the destruction hook completes.

The destruction hook MUST return `void` and MUST NOT be invoked explicitly by ordinary program code. It is a compiler-invoked destruction hook rather than an ordinary callable function. The compiler invokes it exactly once for each

owned value whose type defines one.

If destruction is interrupted by abnormal termination, the language does not require a particular recovery mechanism. Any remaining destruction obligations are handled according to the termination semantics defined by the applicable execution model.

If a type has no destruction hook, its owned fields are recursively destroyed according to their own destruction semantics.

```
T.destroy()
    |
    +-- releases resources managed directly by T
    |   (but not represented by owned fields)
    |
    \-- compiler destroys owned fields
        +-- field A -> its destruction semantics
        +-- field B -> its destruction semantics
        \-- ...
```

Example:

```
struct File
{
    String path;
    raw Handle handle;
}

fn File::destroy(mut File* file)
{
    unsafe
    {
        close_handle(file->handle);
    }
}
```

The intended destruction sequence is:

```
File destroyed
    |
    +-- File::destroy(&mut file)
    |     \-- closes the OS handle
    |
    \-- compiler destroys owned fields
          \-- file.path -> String destruction
```

path is an owned CobaltC field, so File::destroy does not destroy it. The compiler does that afterward.

Meanwhile, the handle is a resource managed directly by File; it is represented as a raw Handle and therefore isn't something the ordinary CobaltC field-destruction machinery would recursively release.

22. Enums

An enum defines a finite set of named variants:

```
enum Status
{
    Ready,
    Running,
    Failed
}
```

Variants MAY contain associated values:

```
enum Result<T,E>
{
    Ok(T),
    Err(E)
}
```

Variant names MUST be unique within the enum. Variants are part of the enum's semantic interface and MAY be used to construct and pattern-match values.

An enum MAY have zero variants. Such an enum has no constructible value and therefore can only be used in contexts that do not require producing a value.

A variant constructor is not an ordinary function. It nevertheless participates in type checking as a value constructor.

23. Arrays

Arrays contain a fixed number of elements and use the type form:

```
T[N]
```

The length N is part of the array type. Array initialization may use an element list:

```
i32[3] values = [10, 20, 30];
```

The number and types of initializer elements MUST match the array type.

A zero-length array T[0] is permitted. It contains no elements and cannot be indexed, but it may be sliced with ... to produce an empty slice.

Safe indexing MUST remain within the valid range.

24. Functions

A function is declared by specifying its name, parameters, and optionally its return type:

```
fn add(i32 a, i32 b) : i32
{
    return a + b;
}
```

If the return type is omitted, the return type is void :

```
fn log(String message)
{
    print(message);
}
```

An explicit void return type is also permitted. A function parameter has the declared type and access capability specified in its declaration. Ownership, borrowing, mutability, and lifetime semantics apply to parameters and return values according to their types and the ordinary rules of this specification.

A non- void function MUST return a value compatible with its declared return type on every successful control-flow path. A void function MUST NOT return a value.

Function names MUST NOT be overloaded within the same unqualified lookup scope. Associated functions are identified separately by their associated type and function name.

Associated Functions

An associated function is declared using the form Type::function . Association provides type-qualified organization and lookup but does not introduce object-oriented method dispatch, an implicit receiver, or an implicit this or self parameter.

```
fn Stack::push(mut Stack* stack, i32 value)
{
    ...
}
```

An associated function is otherwise an ordinary function. All parameters MUST be declared explicitly, and association does not alter the function's parameter list, calling convention, ownership, borrowing, mutability, lifetime, or return-value semantics.

An operation on a type MAY instead be declared as an ordinary function:

```
fn Stack_push(mut Stack* stack, i32 value)
{
    ...
}
```

The ordinary and associated forms have the same fundamental function semantics. They are separate functions; association determines how the function is named and discovered.

Function Parameters and Mutable Access

A parameter of an ordinary value type receives an owned value according to the function-call and ownership rules.

A parameter of type T* provides shared, read-only access to a referent. A parameter of type mut T* provides exclusive mutable access to a referent. A mutable parameter therefore MUST be declared with the mut qualifier on the managed-pointer type when the function is intended to modify the referenced value.

```
fn append_value(mut String* value)
{
    append(*value, "!");
}

mut String text = "hello";
append_value(&mut text);
```

The mut qualifier on a managed-pointer type specifies the parameter's access capability. A separate binding-level mut on the parameter is not required.

When an argument of type mut T* is supplied to a parameter of type T* , the argument undergoes a shared reborrow. This conversion does not move the original mutable managed pointer; the original mutable capability is temporarily restricted while the shared reborrow remains live.

Type-Qualified Associated Functions

An associated function MAY be referenced through its associated type using a type-qualified name:

```
Stack<i32>::new();
Stack<i32>::push(&mut stack, 10);
```

For a generic type, the type arguments form part of the qualification. A type-qualified call does not provide an implicit receiver. The selected function receives only the arguments explicitly supplied by the call.

An associated function is not required to operate on an existing value of its associated type. It MAY create and return a new value:

```
Stack<i32> stack = Stack<i32>::new();
```

new is not a special constructor mechanism. It is an ordinary associated function whose declared return type determines the value produced by the call.

Value-Qualified Associated Functions

An associated function MAY also be referenced through a value whose statically known type is the associated type:

```
mut Stack<i32> stack = Stack<i32>::new();

stack::push(&mut stack, 10);
stack::push(&mut stack, 20);
stack::push(&mut stack, 30);
```

A value-qualified call is not method syntax. The qualifying value determines the type used for associated-function lookup but is not implicitly passed to the selected function.

Consequently, when stack has type Stack<i32> , the following calls identify the same associated function:

```
Stack<i32>::push(&mut stack, 10);
stack::push(&mut stack, 10);
```

In both calls, &mut stack is an explicit argument because the selected function declares a parameter of type mut Stack* .

The :: qualification syntax determines whether associated-function lookup is performed. It does not alter the function's declared parameter list or introduce an implicit receiver. The . operator remains member access and MUST NOT introduce implicit method lookup or invocation.

Associated Function Identity and Name Resolution

An associated function is identified by its associated type and function name. Associated functions with the same name MAY exist for different types and are distinct functions:

```
fn Stack::push(mut Stack* stack, i32 value) { ... }
fn HashTable::push(mut HashTable* table, i32 value) { ... }
```

An ordinary function and an associated function MAY also have the same unqualified function name:

```
fn push(i32 value) { ... }
fn Stack::push(mut Stack* stack, i32 value) { ... }
```

These declarations identify distinct functions. An unqualified call performs ordinary-function lookup:

```
push(10);
```

A type-qualified call performs associated-function lookup for the specified type:

```
Stack<i32>::push(&mut stack, 10);
```

A value-qualified call performs associated-function lookup using the statically known type of the qualifying value:

```
stack::push(&mut stack, 10);
```

The existence of an ordinary function with the same unqualified name MUST NOT cause it to be selected by a type-qualified or value-qualified associated-function call. Likewise, the existence of an associated function MUST NOT cause it to be selected by an unqualified call.

Once an associated function has been selected, the number and types of the explicitly supplied arguments MUST satisfy that function's signature. Ordinary function-call rules, including evaluation order, ownership, borrowing, mutability, lifetime, and type compatibility, then apply.

Modules and External Interfaces

Ordinary functions and associated functions MAY be declared within the same module as the types on which they operate. A module MAY export types and functions through its export { ... } block to form its external interface.

Nothing is exported implicitly. A declaration is externally accessible only when it is exported by the defining module.

Function Semantics

Functions do not acquire special semantics merely because they are associated with a type. Functions remain ordinary callable entities whose ownership, parameter, borrowing, mutability, lifetime, and return-value behavior is determined by their declarations and by the general rules of this specification.

This design permits APIs to be organized around associated types while retaining explicit argument passing and C-like function semantics. In particular, an existing value may be used to select an associated function through value-qualified lookup, but the value is passed to that function only when it is explicitly supplied as an argument.

25. Expressions

Expressions produce values or perform operations. Core expression forms include names, literals, calls, construction, member access, indexing, borrowing, unary operators, binary operators, assignment, postfix error propagation, and range/slice expressions.

Member access uses . . Pointer member access uses -> and is shorthand for dereferencing the pointer and then performing member access:

```
pointer->field
(*pointer).field
```

For raw pointers, the dereference remains an unsafe operation.

Operands are evaluated from left to right. Function arguments are evaluated from left to right before control enters the called function.

26. Operator Precedence

From highest to lowest:

Binary operators are left-associative unless otherwise specified. Assignment is right-associative. :: is name-qualification syntax, not an operator. start..end and start... are slice syntax rather than general binary operators.

&& and || are short-circuiting: the right operand of && is evaluated only when the left operand is true, and the right operand of || is evaluated only when the left operand is false.

27. Arithmetic

Integer and floating-point operations follow the semantics of their respective types.

For integers, an operation whose mathematical result is outside the destination type's range is an arithmetic failure. Such an operation MUST NOT silently wrap, produce an unspecified value, or otherwise produce a value that is treated as a successful result in safe code.

Division by zero is an arithmetic failure. Signed division whose mathematical quotient is not representable by the destination type is also an arithmetic failure.

Shift counts outside the valid range for the left operand's width are arithmetic failures in safe code.

An arithmetic failure that can be determined at compile time MUST be diagnosed as a compile-time error. The implementation MUST NOT generate code that evaluates such an operation as a successful operation.

```
i32 x = 2147483647;
i32 y = x + 1; // ERROR: integer overflow
```

An arithmetic failure that cannot be determined at compile time MUST be detected when the operation is evaluated at runtime. A runtime arithmetic failure MUST NOT produce a value, and execution MUST NOT continue past the failed operation as though it had successfully completed.

```
i32 x = read_i32();
i32 y = x + 1; // May fail at runtime if x is 2147483647
```

The mechanism used to report a runtime arithmetic failure is implementation-defined. An implementation MAY use a checked error, a runtime trap, program termination, or another documented failure mechanism, provided that the mechanism preserves the guarantees of safe code. The implementation MUST document the mechanism it uses.

The implementation-defined reporting mechanism MUST NOT change whether the operation is considered to have failed. Portable programs MUST NOT depend on a particular reporting mechanism, but MAY rely on the guarantees that a failed arithmetic operation produces no value, does not silently succeed, and does not result in undefined memory behavior.

Floating-point operations follow the implementation's documented floating-point model. The model MUST define the behavior of infinities and NaNs if the implementation exposes them.

28. Equality

Equality requires compatible operands. Value equality compares values according to the type's equality semantics.

For floating-point values, equality follows the implementation's documented floating-point model. If NaN values are supported, implementations MUST document whether NaN compares equal to itself; the default CobaltC rule is that NaN is not equal to any value, including itself.

Managed-pointer equality compares pointer identity rather than recursively comparing referents. Pointer identity is the identity of the abstract referent allocation or storage location, not merely a machine address; destruction followed by reuse of an address does not make two different referents identical.

29. Assignment

Assignment requires a valid mutable destination.

For an owned destination, assignment replaces the previous owned value. The previous value is destroyed before the new value becomes the destination's owned value, unless the compiler can preserve the same observable destruction behavior through an equivalent implementation technique.

An assignment of a non-copyable value requires an explicit move . An assignment of a copyable value copies the value unless move is explicitly used.

```
String a = "one";
String b = "two";
a = move b;
```

After a move assignment, the moved-from binding no longer owns the transferred value and cannot be used as an owner except where the partial-move rules permit.

30. Function Calls

A call is valid only if the function is resolvable, the argument count and types match, and ownership, borrowing, and lifetime requirements are satisfied.

Arguments are evaluated left to right. Borrowing and ownership checks apply to the complete argument list before the callee begins execution.

A by-value parameter receives an owned value. If the argument is used directly as a by-value value, a copyable argument is copied and a non-copyable argument is moved. A permitted managed-pointer capability conversion, such as mut T* to T* , is performed instead as a reborrow and is not subject to this copy/move rule.

Once a non-copyable argument is moved into a parameter, the caller no longer owns it. Borrowed parameters do not transfer ownership. A function that wants to return ownership to its caller MUST return the value explicitly as part of its result.

31. Conditional Execution

CobaltC provides conditional execution using:

```
if condition
{
    ...
}
else
{
    ...
}
```

The condition MUST have type bool . An else if chain is equivalent to nested conditionals.

32. Loops

CobaltC provides for , foreach , while , and loop loops.

Conditional loop forms use parenthesized conditions. Every loop must have a block body enclosed in braces. A semicolon or other statement cannot be used as a loop body.

```
for (initializer; condition; increment) { ... }
foreach (value in sequence) { ... }
while (condition) { ... }
loop { ... }
```

A for loop consists of an initializer, a condition, and an increment expression. The initializer is evaluated once before the first iteration. Before each iteration, the condition is evaluated. If the condition is false, the loop terminates. After each iteration of the loop body, the increment expression is evaluated before the next condition evaluation.

The initializer, condition, and increment expression are optional. An omitted condition is treated as always true.

```
for (;;) { ... }
```

A semicolon immediately following a loop header is not a valid loop body and is a syntax error.

```
for (;;); // invalid
for (;;); { ... } // invalid
```

A foreach loop iterates over the values of a range or other language-defined sequence. The exact iteration mechanism is implementation-defined where this specification does not otherwise constrain it.

```
foreach (value in sequence) { ... }
```

A loop body is a scope. Deferred blocks registered in an iteration are executed when control leaves that body, including when break or continue leaves the body.

```
while (condition)
{
    Resource resource = acquire();
    defer { release(resource); };
    if (done) { break; }
}
```

In this example, the deferred block executes when the loop body is exited, including when break leaves the loop.

break exits the innermost applicable loop. continue begins the next iteration of the innermost applicable loop. In a for loop, continue evaluates the increment expression before the next condition evaluation.

A deferred block is associated with the innermost enclosing scope and executes when control leaves that scope, regardless of whether the exit occurs through normal completion, return, break, continue, or another control transfer.

33. Match

match may be used as a statement or expression.

```
match status {
    Ready => use_ready(),
    Running => use_running(),
    Failed => use_default()
}
```

A match expression may produce a value:

```
String result = match value {
    1 => "one",
    2 => "two",
    _ => "other"
};
```

Match arms are evaluated in source order and the first matching arm is selected. The _ pattern matches any value not matched by an earlier arm. The compiler MUST reject statically non-exhaustive matches over a finite set of known variants or otherwise exhaustively enumerable cases.

34. Return

return transfers control from the current function.

The return expression is evaluated before deferred blocks and local destruction associated with the exited scopes. The resulting value is then transferred to the caller according to the ownership and borrowing rules.

Returning an owned value transfers ownership to the caller. Returning a managed pointer is permitted only when its lifetime remains valid after the function returns. A managed pointer to an ordinary local variable MUST NOT be returned.

35. Defer

defer registers a block to be executed when the enclosing lexical scope is exited.

```
fn process_file(Path path)
{
    File file = File::open(path);

    defer
    {
        File::close(file);
    };

    process(file);
}
```

A deferred block is associated with the scope in which it is registered. When control leaves that scope, its deferred blocks execute before the scope's owned locals are destroyed. Deferred blocks registered in nested scopes are associated with their respective scopes and execute when those scopes are exited.

```
{
    Resource resource = acquire();

    defer
    {
        release(resource);
    };

    use(resource);
}
```

Deferred blocks execute in reverse registration order within each scope. For nested scopes, deferred blocks in the inner scope execute before deferred blocks in the enclosing scope.

```
{
    defer
    {
        log("outer");
    };
    {
```

```
        defer
        {
            log("inner");
        };
    }
}

// Output:
// inner
// outer
```

A deferred block observes the current values of its referenced bindings at the time it executes. Registering a deferred block therefore creates a future use of each referenced binding. A referenced binding MUST remain initialized and otherwise valid until the deferred block has executed.

```
String value = "hello";

defer
{
    print(value);
};

value = "goodbye";

// Prints: goodbye
```

A binding referenced by a deferred block MUST NOT be moved, destroyed, or otherwise rendered invalid before the deferred block executes.

```
String value = "hello";

defer
{
    print(value);
};

String other = move value; // ERROR: value is required by the deferred block
```

Deferred blocks execute before the destruction of owned locals in their associated scope. Consequently, a deferred block MAY access an owned local that would otherwise be destroyed when the scope is exited.

A deferred block MUST NOT access a binding whose lifetime ends before the deferred block executes. If such access would occur, the program is ill-formed.

36. Definite Initialization

A value MUST be initialized before it is read.

The compiler MUST perform control-flow-sensitive definite-initialization analysis sufficient to establish whether every path reaching a read has initialized the value.

```
i32 value;
if condition
{
    value = 10;
}
print(value);
```

The example is invalid unless the compiler can prove that every path reaching print initializes value .

37. Ownership

Ownership is a fundamental part of CobaltC's type and runtime model. An owned value has exactly one responsible owner, and that owner is responsible for eventual destruction.

Managed pointers are non-owning. Library abstractions MAY implement shared ownership, but such ownership MUST be explicit in the abstraction and is not inferred from an ordinary managed pointer. Shared ownership implemented by a library is

an abstraction-level resource-management mechanism, not an additional ordinary ownership state of the language.

Ownership transfer is explicit through moves and through operations whose signatures consume owned values. Passing a copyable value by value copies it; passing a non-copyable value by value moves it.

A value is copyable only when its type's copy contract permits copying. The predefined value types and shared managed pointers are copyable unless otherwise stated. A user-defined aggregate is copyable when all of its owned components are copyable and the type does not define a destruction hook. Types provided by the standard library MAY define their own copy contracts. A mutable managed pointer is not copyable. A copy operation produces an independent value according to the type's copy contract.

38. Move Semantics

A move transfers ownership of an owned value. Moving a managed pointer value transfers its borrow capability rather than ownership of its referent.

```
File a = open("data.txt")?;
File b = move a;
```

After the move, a remains a binding but no longer owns the transferred value. It MUST NOT be read, moved, or destroyed as though it still owned that value. The moved-from binding may be reassigned with a newly initialized value.

39. Copy Semantics

A type may support copying. Copying is implicit when a copyable value is used in a by-value assignment, initialization, or function argument position and an explicit move is not present.

Copying produces an independent value according to the type's copy contract. Copying is not ownership transfer.

A shared managed pointer is copyable; copying it creates another shared borrow. A mutable managed pointer is not implicitly copyable.

40. Partial Moves

For aggregate values, an individual owned component MAY be moved independently when the resulting ownership state is well-defined according to the language rules.

After a component is moved, that component is considered moved from the containing aggregate. The containing aggregate remains usable through components that remain owned and initialized. A moved component MUST NOT subsequently be accessed through its original ownership path until it is reinitialized.

```
struct Person
{
    String name;
    String address;
}

Person person = ...;

String name = move person.name;

print(person.address); // OK
print(person.name);    // ERROR: name was moved
```

An aggregate MUST NOT be used in an operation that requires ownership or initialization of a moved component. In particular, an aggregate with a moved component MUST NOT be moved, copied, returned, or passed by value when doing so would require that component to be available.

```
Person person = ...;
```

```
String name = move person.name;

Person other = move person; // ERROR: person.name has been moved
```

A moved component MUST NOT be destroyed again through the original aggregate ownership path. If the aggregate later reaches destruction, only the components that remain owned and initialized by that aggregate are destroyed.

```
Person person = ...;

String name = move person.name;

// `person.name` is no longer owned by `person`.
// When `person` is destroyed, only `person.address` is destroyed.
```

A moved component MAY be reinitialized through its original ownership path when the language rules permit assignment to an uninitialized component. Once reinitialized, the component is again owned and initialized by the containing aggregate and participates in its subsequent destruction.

```
Person person = ...;

String name = move person.name;

person.name = "new name"; // OK: name is reinitialized

// `person` now owns both name and address again.
```

41. Borrowing

A value MUST NOT be destroyed, reassigned, replaced, or otherwise invalidated while a live borrow of that value or overlapping storage would be invalidated by the operation. A borrow provides access to an owned value without transferring ownership. Borrowing does not create a new owner of the borrowed value.

CobaltC supports shared borrows and mutable borrows. A shared borrow provides read-only access to its referent. A mutable borrow provides exclusive mutable access to its referent.

The managed pointer type T* represents a shared, non-null managed pointer. The managed pointer type mut T* represents an exclusive, mutable, non-null managed pointer.

The expression &expr creates a shared borrow of type T* . The expression &mut expr creates a mutable borrow of type mut T* when the applicable ownership, mutability, and borrowing rules permit the operation.

The fundamental borrowing rule is:

```
zero or more compatible shared borrows OR one mutable borrow
```

Multiple compatible shared borrows MAY exist simultaneously. A shared borrow is compatible with another shared borrow when neither borrow provides conflicting access to the same storage.

```
String value = "hello";
String* pointer = &value;
print(*pointer);
print(value);
```

The example is valid because both accesses are read-only shared access. The borrow does not transfer ownership of value . The binding value remains the owner of the string.

A mutable borrow is incompatible with any other borrow of the same storage that would permit conflicting access. Conflicting borrows MUST be rejected.

```
mut String value = "hello";
String* shared = &value;
print(*shared);
mut String* mutable = &mut value; // ERROR: conflicting borrow
```

A value MUST NOT be moved while a live borrow of that value would be invalidated by the move.

```
String value = "hello";
String* pointer = &value;
```

```
String moved = move value;
print(*pointer); // ERROR: `value` was moved while borrowed
```

A borrow MUST NOT outlive its referent. A managed pointer MUST NOT be used after the storage required by that pointer has ceased to be valid.

42. Shared Borrows

A shared borrow provides read-only access to its referent. Multiple compatible shared borrows MAY exist simultaneously.

```
String value = "hello";
String* first = &value;
String* second = &value;
print(*first);
print(*second);
```

Shared borrows MAY alias the same value when they provide only compatible shared access.

```
String value = "hello";
String* first = &value;
String* second = &value;
String* third = &value;
print(*first);
print(*second);
print(*third);
```

A shared borrow MUST NOT be used to perform mutable access to its referent.

```
String value = "hello";
String* pointer = &value;
append(*pointer, "!"); // ERROR: shared borrow does not permit mutation
```

A mutable borrow MUST NOT be created while an incompatible shared borrow remains live.

```
mut String value = "hello";
String* shared = &value;
print(*shared);
mut String* mutable = &mut value; // ERROR: `value` is still borrowed
append(*mutable, "!");
```

The compiler MUST permit multiple compatible shared borrows and MUST reject conflicting mutable access.

43. Mutable Borrows

A mutable borrow provides exclusive mutable access to its referent.

A mutable borrow requires a mutable owner or otherwise mutable storage as defined by the applicable type rules.

```
mut String value = "hello";
mut String* pointer = &mut value;
append(*pointer, " world");
print(*pointer);
```

While a mutable borrow is live, another mutable borrow of overlapping storage MUST NOT be created.

```
mut String value = "hello";
mut String* first = &mut value;
mut String* second = &mut value; // ERROR: conflicting mutable borrow
append(*first, "!");
append(*second, "?");
```

A mutable borrow MUST NOT coexist with a conflicting shared borrow.

```
mut String value = "hello";
String* shared = &value;
mut String* mutable = &mut value; // ERROR: conflicting borrow
print(*shared);
append(*mutable, "!");
```

A mutable managed pointer MAY subsequently be reborrowed or converted to a shared managed pointer when the resulting shared access satisfies the applicable borrowing and lifetime rules.

```
mut String value = "hello";
mut String* mutable = &mut value;
append(*mutable, "!");
String* shared = mutable;
print(*shared);
```

The resulting shared pointer provides only shared access. The mutable capability MUST NOT be used in a conflicting manner while the shared reborrow remains live.

Mutable access MUST remain exclusive for the duration of the applicable mutable borrow.

44. Borrow Lifetime

A borrow has a lifetime during which its managed pointer remains valid and its borrowing restrictions apply.

A borrow is live at a program point if an execution path from that point may subsequently access the corresponding managed pointer or perform an operation whose validity depends on that borrow.

A borrow MUST NOT outlive its referent. A borrow remains live for at least the period required by all later uses of that borrow. An implementation MAY determine that a borrow is no longer required earlier, provided doing so cannot violate borrowing rules or change observable program behaviour.

```
String value = "hello";
String* pointer = &value;

print(*pointer);

// `pointer` is no longer used, so the borrow may end here.
doSomethingWith(value); // OK
```

A managed pointer passed as an ordinary by-value argument remains live for the duration of the call.

```
fn use_value(String* value)
{
    print(*value);
}

String value = "hello";
use_value(&value); // borrow remains valid for the duration of the call
```

A function may return a managed pointer derived from an input borrow only when the result explicitly derives from that borrow and its lifetime does not outlive the input borrow.

```
fn first(String* value) : String*
{
    return value;
}

String value = "hello";
String* pointer = first(&value);

print(*pointer); // OK: pointer does not outlive value
```

A returned managed pointer MUST NOT outlive the borrow from which it was derived.

```
fn get_value() : String*
{
    String value = "hello";
    return &value; // ERROR: returned borrow would outlive value
}
```

A borrow MUST NOT outlive its referent.

```
String* pointer;
{
    String value = "hello";
    pointer = &value;
}
```

```
print(*pointer); // ERROR: referent no longer exists
```

The implementation MAY use conservative lifetime analysis. It MUST NOT accept a use that would outlive the referent.

45. Function Parameters and Returned Borrows

A function may accept a managed pointer without taking ownership of the referent. A parameter of type T* provides shared access; mut T* provides exclusive mutable access.

When an argument of type mut T* is supplied where T* is required, the language performs a shared reborrow. This conversion does not move the original mutable capability; the original capability is temporarily restricted while the reborrow remains live.

```
fn length(String* value) : usize
{
    return length_of(*value);
}
```

A function MAY return a managed pointer when the returned pointer remains valid after the function returns. A returned borrow derived from an input borrow MUST NOT outlive that input borrow.

```
fn identity_borrow(String* value) : String*
{
    return value;
}
```

A compiler MUST reject a returned borrow when it cannot establish the required lifetime relationship. A managed pointer to an ordinary local variable MUST NOT be returned.

46. Reborrowing

A managed pointer MAY be used to create another borrow of the storage it refers to. Such an operation is a reborrow.

A reborrow creates a new borrow whose access capability is derived from the existing managed pointer. Reborrowing MUST preserve the ownership, lifetime, and aliasing guarantees of the original borrow.

A reborrow operates on the storage referred to by the managed pointer; it does not move or transfer ownership of the managed pointer itself.

A shared managed pointer MAY be reborrowed as another shared managed pointer. Such a reborrow creates another shared borrow of the same storage.

A mutable managed pointer MAY be reborrowed as a mutable managed pointer, provided no conflicting borrow of the same storage is live.

```
fn append_exclamation(mut String* value)
{
    append(*value, "!");
}

mut String value = "hello";
mut String* pointer = &mut value;

// Reborrow the referent mutably for the duration of the call.
// &mut *pointer does not move pointer. The original mutable capability
// may be used again after the reborrow is no longer live.

// The temporary mutable reborrow ends when the call returns.
append_exclamation(&mut *pointer);

append(*pointer, "?");
```

A mutable managed pointer MAY also be reborrowed as a shared managed pointer when the resulting shared borrow satisfies the applicable lifetime and aliasing rules. Such a reborrow does not move or consume the original mutable capability.

```
mut String value = "hello";
mut String* pointer = &mut value;

// *pointer means "dereference the pointer and access the referent".
// The operation below borrows the referent, not the pointer variable.
// Taking a borrow of the dereferenced value is a valid reborrow:
String* shared = & *pointer;

print(*shared);

// `pointer` MUST NOT be used for conflicting mutable access while
// the shared reborrow remains live.
```

A reborrow creates a new managed pointer capability to the same referent. It does not create ownership, transfer ownership of the underlying value, or change destruction responsibility.

```
mut String value = "hello";
mut String* pointer = &mut value;

// Taking a mutable borrow of the dereferenced value creates a mutable reborrow:
mut String* reborrow = &mut *pointer;

append(*pointer, "!"); // ERROR: conflicts with live mutable reborrow
append(*reborrow, "?");
```

A shared reborrow derived from a mutable borrow temporarily restricts the originating mutable capability. While the shared reborrow is live, the originating mutable capability MUST NOT be used for conflicting access.

Once the reborrow is no longer live, the originating mutable capability MAY be used again, subject to the ordinary borrowing rules.

47. Field and Partial Borrows

A field of a structure MAY be borrowed independently of another disjoint field.

```
struct Pair
{
    String first;
    String second;
}

mut Pair pair = Pair { first = "one", second = "two" };
mut String* first = &mut pair.first;
mut String* second = &mut pair.second;
append(*first, "!");
append(*second, "?");
```

A borrow of one field does not, by itself, prevent access to a disjoint field.

```
mut Pair pair = Pair { first = "one", second = "two" };
mut String* first = &mut pair.first;
append(pair.second, "!");
append(*first, "?");
```

Two field paths are disjoint when they identify distinct, non-overlapping storage within the same aggregate value.

The compiler MUST permit simultaneous borrows of fields that are established to be disjoint.

The compiler MUST reject simultaneous mutable borrows when the borrowed field paths may refer to overlapping storage.

```
mut Pair pair = Pair { first = "one", second = "two" };
mut Pair* whole = &mut pair;
mut String* first = &mut pair.first;
use(*whole); // ERROR: conflicting borrow
```

A borrow of an entire aggregate conflicts with a mutable borrow of any overlapping part of that aggregate.

Partial borrowing MUST preserve the same aliasing, lifetime, and exclusivity guarantees as borrowing an entire value.

48. Aliasing

Aliasing occurs when more than one managed pointer provides access to the same underlying storage.

Multiple compatible shared managed pointers MAY alias the same storage.

```
String value = "hello";
String* first = &value;
String* second = &value;
print(*first);
print(*second);
```

A mutable managed pointer is exclusive. A mutable managed pointer MUST NOT coexist with another managed pointer that permits conflicting access to the same storage.

```
mut String value = "hello";
mut String* first = &mut value;
mut String* second = &mut value; // ERROR: mutable aliases are prohibited
append(*first, "!");
append(*second, "?");
```

For any storage location accessible through managed pointers, CobaltC MUST enforce the following invariant:

```
multiple compatible shared managed pointers OR one mutable managed pointer
```

A mutable managed pointer MUST NOT coexist with a conflicting shared managed pointer. Two mutable managed pointers MUST NOT provide conflicting access to the same storage.

The following is therefore permitted when all managed pointers provide compatible shared access:

```
shared shared shared
```

The following are prohibited when the managed pointers provide conflicting access to the same storage:

```
mutable mutable
shared mutable
```

These aliasing requirements apply to direct managed pointers, reborrows, field borrows, function parameters, returned borrows, and collection element borrows.

Safe CobaltC operations MUST NOT provide a means to bypass these aliasing requirements.

49. Collection Borrowing

Elements of a collection MAY be borrowed when the collection operation and element type permit the corresponding access.

```
Vector<i32> values = [10, 20, 30];
i32* first = &values[0];
i32* second = &values[1];
```

A mutable element may be borrowed when the collection and element permit mutable access.

```
mut Vector<i32> values = [10, 20, 30];
mut i32* first = &mut values[0];
*first = *first + 1;
```

A collection operation that requires mutable access or may relocate storage MUST NOT occur while a conflicting element or collection borrow remains live, unless the compiler can establish that the referenced storage is unaffected.

For dynamic indexes, the implementation MAY conservatively treat potentially overlapping accesses as conflicting. It MUST NOT assume disjointness merely because different runtime indexes are expected.

50. Borrow Invalidation

A managed pointer is valid only while its referent remains valid and the pointer satisfies the applicable borrowing rules.

An operation that conflicts with a live borrow MUST be rejected. A conflicting operation does not by itself make an otherwise valid managed pointer safe to use; the operation is prohibited while the conflicting borrow remains live.

A managed pointer MUST NOT be used after its referent has ceased to exist or after storage required by that pointer has ceased to be valid.

```
String* pointer;
{
    String value = "hello";
    pointer = &value;
}
print(*pointer); // ERROR: referent has been destroyed
```

A value MUST NOT be moved while a live borrow of that value would be invalidated by the move.

```
String value = "hello";
String* pointer = &value;
String moved = move value;
print(*pointer); // ERROR: `value` was moved while borrowed
```

A borrow MAY cease to restrict a value once the corresponding managed pointer is no longer used and the borrow is therefore no longer live.

```
mut String value = "hello";
{
    String* pointer = &value;
    print(*pointer);
}
append(value, " world");
print(value);
```

A collection operation that could invalidate an active element borrow MUST be rejected while that borrow remains live.

```
mut Vector<String> values = ["hello"];
String* pointer = &values[0];
values::push(&mut values, "world"); // ERROR: active element borrow
print(*pointer);
```

The compiler MUST reject a program when it can establish that a managed pointer would be used after its referent becomes invalid, or when an operation would violate the shared-borrow, mutable-borrow, lifetime, move, aliasing, or collection-borrowing rules.

Collection Invalidation and Conservative Analysis

An operation that may reallocate, resize, relocate, or otherwise invalidate storage MUST NOT be performed while a live managed pointer or slice depends on that storage, unless the compiler can establish that the particular reference is unaffected.

An implementation MAY use static or runtime techniques to prove that a particular operation cannot invalidate a particular reference. Safe behavior MUST be preserved regardless of whether the implementation uses compile-time or runtime analysis.

51. Destruction

Owned values are destroyed deterministically according to Section 21. An ownership responsibility is destroyed exactly once. Moved-from ownership does not cause a second destruction.

The compiler provides an internal drop operation for performing destruction of an owned value. The drop operation is a compiler-level operation and is not ordinary source-language syntax.

When the compiler performs drop on an owned value, the value is destroyed according to its destruction semantics and the ownership responsibility for that value is consumed.

If the value's type defines a destroy hook, that hook is invoked according to Section 21. Otherwise, the compiler recursively destroys the value's owned fields according to their destruction semantics.

Ordinary program code MUST NOT invoke the compiler-level drop operation explicitly.

52. Scope Destruction

For ordinary scope exit:

deferred blocks execute;

owned locals are destroyed in reverse declaration order;

control proceeds to the enclosing scope.

An implementation MUST preserve the observable consequences of this ordering.

53. Unwinding

If the implementation supports unwinding, scopes exited by supported unwinding MUST perform their specified destruction.

Unwinding Boundary

CobaltC does not define panic or exception unwinding as a source-language mechanism. An implementation MAY use internal unwinding or another mechanism for its own purposes, provided that it does not introduce source-language semantics or observable behavior beyond those already specified by this document.

54. Abort

An abort terminates execution immediately.

Normal destruction is not guaranteed after an abort.

55. Nullability

Nullable managed pointers are explicitly represented by nullable pointer types.

null cannot inhabit a non-nullable managed pointer type.

Before dereferencing a nullable managed pointer, non-nullness MUST have been established by a test or equivalent language rule. Flow-sensitive refinement is permitted:

```
String*? value = find();
if (value != null)
{
    print(*value);
}
```

A refinement remains valid until the refined binding is assigned or another operation can change the condition on which the refinement depends. Passing the pointer by ordinary value does not itself invalidate the refinement.

56. Bounds Safety

Safe indexing MUST remain within valid bounds.

The compiler MAY eliminate runtime bounds checks when validity has been proven statically.

Unchecked indexing belongs to unsafe facilities.

The compiler MAY eliminate a runtime bounds check when it can prove statically that the index is valid. The optimization MUST preserve the observable semantics required by safe indexing. Unchecked indexing remains an unsafe facility.

57. Result<T,E>

For terminology consistency, CobaltC distinguishes between an error value represented by a program-visible type such as Result<T,E> , a runtime failure in which an operation cannot successfully produce its specified result, and abnormal termination mechanisms such as abort or unwinding. These mechanisms are distinct and are governed by their respective sections.

The canonical recoverable-error type is:

```
enum Result<T,E>
{
    Ok(T),
    Err(E)
}
```

Expected operational failures SHOULD be represented using Result . The variants Ok and Err are ordinary enum variants and may be constructed and matched like other variants.

58. Error Propagation

The postfix ? operator propagates a compatible error from the current operation to the enclosing function.

```
fn load() : Result<String, IoError>
{
    String text = read_file("data.txt")?;
    return Ok(text);
}
```

The operator evaluates its operand. If the operand is a successful Result , the contained value continues the expression. If it is an Err , that error is returned from the current function.

The operand of ? MUST have an error-propagation form defined by this section, and the propagated error MUST be compatible with the enclosing function result. ? is not an exception mechanism and does not implicitly unwind scopes other than the ordinary control-flow consequences of returning.

59. Strings

String is an owned text type that owns its UTF-8 storage.

A valid text value MUST contain valid UTF-8. Arbitrary byte sequences are not text values and require byte-oriented types or APIs.

A string literal has type String . A String MAY be borrowed through the ordinary shared-borrow rules, producing a shared managed pointer to the string.

```
String message = "hello, CobaltC";
String* view = &message;

print(*view);
```

In this example, message remains the owner of the string storage and view provides borrowed read-only access to that storage. The borrow MUST remain valid for the entire period in which view is used.

60. Vector<T>

Vector<T> owns dynamically allocated contiguous storage.

Its capacity MAY exceed its current length.

Operations that change storage in ways that could invalidate active references are governed by the borrowing rules.

A Vector<T> owns its elements according to the ownership semantics of T . Borrowed access to vector elements does not transfer ownership of the elements or the underlying storage.

Storage Invalidation

Because a Vector may relocate its contiguous storage, any operation capable of changing storage in a way that would invalidate an active element reference is subject to the borrow-invalidation rules of Section 50. An implementation MAY avoid relocation in a particular case, but it MUST NOT allow an active managed pointer to become invalid while remaining usable.

61. Slices

A slice provides borrowed access to a contiguous range of elements in another value. It does not own the elements or storage.

A slice remains valid only while the source storage remains valid and while no operation invalidates the referenced range according to the borrowing rules. A slice does not extend the lifetime of the value from which it was created.

CobaltC slice ranges use inclusive endpoints. Unlike half-open range conventions used by some languages, both the starting and ending indices identify elements included in the resulting slice:

```
values[0..3]
values[0...]
values[...]
```

0..3 includes indices 0 through 3.

0... continues from index 0 through the final valid element.

... selects the entire sequence.

For a non-empty sequence of length n , an explicit inclusive range MUST satisfy 0 <= start <= end < n . A range whose explicit endpoints do not identify valid elements is a bounds error.

The complete-range form ... is valid for an empty sequence and produces an empty slice.

A slice type is written Slice[T] . For slices, binding mutability is explicitly part of the slice access model: a Slice[T] provides shared read-only access, while a mut Slice[T] provides mutable access to the borrowed elements. This is a language-defined exception to the general distinction between binding mutability and managed-pointer capability described in Section 16.

```
Slice[i32] view = values[0..3];
mut Slice[i32] view = values[0..3];
```

Slice access is subject to the same borrowing, aliasing, lifetime, and invalidation requirements as other borrowed access to the underlying storage. A slice's lifetime cannot exceed the lifetime of the storage it refers to. Passing a slice to a function passes borrowed access, not ownership.

62. Threads

CobaltC permits implementations to provide concurrent execution through threads. Thread creation and management are provided by the runtime or standard library rather than being fully defined by the core language.

Values and managed pointers transferred to another thread MUST satisfy all applicable ownership, borrowing, lifetime, and concurrency requirements. A thread boundary MUST NOT implicitly transfer ownership, extend a borrow, or extend the lifetime of a local value.

An ordinary borrow of a local value MUST NOT be transferred to another thread when the referent may cease to exist before all uses of the transferred borrow have ended.

A thread entry operation receives an owned value or other explicitly specified argument capabilities. The mechanism used to transfer those arguments and the capabilities permitted across a thread boundary are implementation-defined, subject to the ownership, borrowing, and lifetime guarantees of this specification.

The core language does not define a general thread-sharing or atomic-memory model. Implementations that provide shared mutable state, synchronization primitives, or atomic operations MUST document their applicable concurrency semantics and MUST NOT weaken the memory-safety guarantees of safe CobaltC code.

63. Synchronization

Implementations that provide shared mutable state or concurrent access to shared values MUST provide synchronization mechanisms appropriate to their documented concurrency semantics.

Programs that access shared mutable state concurrently MUST use synchronization or atomic operations as required by the applicable concurrency semantics. Concurrent access MUST NOT violate the ownership, borrowing, lifetime, or concurrency requirements of this specification.

A synchronization mechanism MUST NOT implicitly transfer ownership, permit an otherwise invalid borrow to cross a thread boundary, extend a borrow, or extend the lifetime of a value.

Synchronization does not alter the destruction rules of a value. In particular, preventing concurrent access to a value does not by itself keep that value alive or make a borrow valid after the value would otherwise cease to exist.

The ordering, visibility, atomicity, and other concurrency guarantees provided by synchronization mechanisms are defined by the language or standard library where specified, and otherwise MUST be documented by the implementation providing those mechanisms.

64. Mutex

A mutex provides mutual exclusion for access protected by the mutex. Concurrent access to protected shared state MUST conform to the concurrency semantics specified for the mutex by the language or standard library.

A lock guard represents an acquired mutex. While a guard is live, the guard is responsible for retaining the lock. Destroying the guard releases the lock.

Lock acquisition blocks until the mutex can be acquired unless the standard-library API explicitly provides a non-blocking operation. A mutex MUST NOT be destroyed while a live guard refers to or otherwise depends on that mutex.

A lock guard is moveable but not copyable. Moving a guard transfers responsibility for releasing the lock to the destination guard.

A mutex or lock guard does not by itself transfer ownership, extend a borrow, extend the lifetime of a value, or alter the destruction rules of protected state.

65. Data Races

Safe CobaltC code MUST NOT contain an ordinary unsynchronized data race.

An ordinary data race occurs when two or more execution contexts access the same mutable memory concurrently, at least one access is a write, and the accesses are not ordered by a synchronization mechanism defined by this specification or a conforming standard-library synchronization facility.

Concurrent access to immutable data does not constitute a data race provided the data remains immutable for the duration of those accesses. Accesses to distinct memory locations do not constitute a data race.

The language does not guarantee freedom from deadlocks, livelocks, starvation, or logical errors in otherwise synchronized programs.

66. Memory Model

The CobaltC memory model defines observable ordering and visibility guarantees for evaluations that access memory.

Within a single execution context, evaluations are sequenced in program order unless a language rule explicitly specifies otherwise.

Conflicting ordinary accesses to the same memory location MUST NOT occur concurrently unless ordered by synchronization.

A successful unlock of a Mutex synchronizes with a subsequent successful lock of the same Mutex . Effects on protected state that occur before the unlock MUST be observable to the execution context after the subsequent successful lock, subject to the ordinary visibility of that state.

Thread start and successful join also establish the synchronization required by their respective runtime operations.

67. Unsafe Code and Functions

Unsafe operations require an explicit unsafe context, established either through an unsafe block or an unsafe function declaration:

```
unsafe
{
    ...
}
```

```
unsafe fn some_function()
{
    ...
}
```

Marking a function as unsafe permits it to contain unsafe operations and indicates to callers that calling the function requires an enclosing unsafe context or block.

Unsafe permits operations requiring programmer-supplied invariants.

It does not make an invalid operation intrinsically correct.

Unsafe Context Boundary

An unsafe context permits operations for which the programmer is responsible for maintaining additional invariants. It does not alter the meaning of safe CobaltC elsewhere in the program and MUST NOT be used to make safe code silently lose its ownership, lifetime, nullability, bounds, or concurrency guarantees.

68. Raw Memory

Raw-pointer dereference, unchecked memory manipulation and manual allocation/deallocation are unsafe facilities.

An implementation MUST NOT treat arbitrary raw memory as automatically satisfying CobaltC's type, lifetime or ownership requirements.

69. Safe Abstractions over Unsafe Code

Unsafe implementation code MAY be encapsulated by a safe API.

Such an API is valid only if its implementation maintains all invariants promised by its safe interface.

Unsafe implementation techniques MAY be encapsulated behind safe interfaces only when those interfaces preserve the invariants promised by their documented types and operations. Internal unsafe mechanisms do not automatically become part of the safe-language semantics.

70. Foreign Functions

Foreign functions require explicit declarations.

The baseline foreign ABI is the C ABI.

Foreign functions are not assumed to obey CobaltC ownership, lifetime or safety rules.

Ownership transfer across a foreign-function boundary MUST be expressed by the declared interface contract. Calling a foreign function does not implicitly transfer ownership, extend a lifetime, or create a managed borrow. Values returned from foreign functions have only the ownership and validity guarantees explicitly established by the declaration or foreign interface rules.

Foreign ABI Baseline

The baseline foreign ABI is the platform's C ABI as described by the applicable ABI profile. An extern declaration and its foreign-call behavior MUST conform to that profile. ABI compatibility alone does not determine CobaltC ownership semantics.

71. FFI Ownership

Ownership crossing an FFI boundary MUST be defined by the API contract.

Possible contracts include:

borrowed for call duration

caller transfers ownership

callee transfers ownership

caller retains ownership

foreign runtime owns value

The ABI alone does not determine ownership.

72. ABI Profiles

A target ABI profile specifies at minimum:

architecture

operating system

pointer width

endianness

alignment

calling conventions

C ABI mapping

concurrency and atomic capabilities of the target platform

runtime model

Binary compatibility is guaranteed only where compatible ABI profiles are used.

Representation and ABI Independence

Where aggregate layout is externally observable, such as at an FFI boundary, it is determined by the applicable ABI profile. An implementation MUST document implementation-defined layout properties where the selected profile requires them. Internal aggregate representation remains an implementation choice unless this specification or an ABI profile makes a property observable.

Managed-reference representation remains an implementation choice so long as all language-level pointer semantics are preserved.

73. Runtime

A hosted CobaltC program begins through an entry point named main , according to the applicable runtime conventions. The language does not otherwise prescribe the startup or process-integration mechanism.

The runtime provides the facilities required by the language and standard library, including:

allocation;

destruction;

process integration;

I/O;

concurrency;

platform integration.

The internal runtime architecture is implementation-defined.

74. Allocation

Managed allocation either produces a valid allocation or produces the specified allocation failure.

Allocation failure MUST NOT expose an invalid managed object. Expected allocation failure SHOULD be represented through the applicable Result -style API.

An implementation MAY impose documented resource limits. Such limits do not permit creation of invalid managed objects.

75. Standard I/O

Examples in this specification MAY use representative standard-library operations to illustrate language semantics. Unless an operation is explicitly specified by a section of this document, its complete declaration and API contract are part of the applicable standard-library profile.

The Standard-conforming library provides basic text and resource I/O operations, including print , println , open , read , write , and close .

Expected I/O failures are represented using Result -style APIs.

A resource-owning I/O object releases its resources deterministically according to the normal destruction rules. An explicit resource-release operation MUST leave the owning object in a valid state such that subsequent destruction does not release the same external resource again.

The exact concrete error and stream types are part of the selected standard-library profile; the operation names and error-reporting model described here are normative for Standard Conformance.

76. Security and Safety Boundary

CobaltC's safety guarantees apply to conforming safe code.

They do not guarantee:

algorithmic correctness;

absence of deadlocks;

absence of resource exhaustion;

absence of denial-of-service conditions;

correctness of unsafe code;

correctness of foreign code;

correctness of violated API preconditions.

77. Diagnostics

A conforming compiler MUST reject programs violating normative static rules.

Diagnostic categories include:

```
syntax error
name-resolution error
type error
initialization error
ownership error
use-after-move
borrow conflict
lifetime violation
nullability violation
bounds violation
non-exhaustive match
invalid assignment
```

Exact diagnostic wording is not normative.

Implementations SHOULD identify relevant source locations and, where practical, explain ownership and lifetime relationships.

A compiler MAY continue parsing after a syntax error to report additional diagnostics. Error recovery MUST NOT cause the compiler to accept a program that violates a normative syntax or semantic rule. Recovery boundaries such as ; , } , or declaration keywords do not themselves introduce additional language keywords or syntax.

78. Implementation-Defined Behavior

Any implementation-defined language property MUST be documented as part of the implementation's conformance declaration or target profile.

Implementation-defined choices MAY include representation details, implementation limits, floating-point facilities, and platform integration where this specification permits such choices.

Implementation techniques such as region inference, constraint solving, monomorphization, pointer-based representations, compiler-generated destruction, or generated synchronization are not language semantics merely because an implementation uses them.

79. Extensions

An implementation MAY provide extensions. Extensions MUST be distinguishable from standard CobaltC behavior by an explicit opt-in mechanism such as a compiler option, module, namespace, or extension-specific syntax.

An extension MUST NOT silently change the semantics of a valid CobaltC 1.0.1 program.

CobaltC 1.0.1 does not define a general-purpose macro-expansion system. Preprocessing, code generation, templates, and source transformation MAY be provided as extensions subject to the same rule.

80. Conformance Levels

Core Conformance

Requires the language syntax, type system, static semantics, ownership, borrowing, lifetimes, and core safety guarantees.

Standard Conformance

Requires Core plus the mandatory standard-library baseline: Result , String , Vector , Slice , Mutex , and Standard I/O. Implementations MUST provide the source-visible facilities described by those sections.

Platform Conformance

Requires Standard plus a complete declared runtime and ABI profile for the target platform.

An implementation claiming conformance MUST state its level and MUST document any implementation-defined choices required by that level.

81. Conformance Testing

A conformance suite SHOULD contain positive and negative tests covering lexing, parsing, name resolution, typing, initialization, ownership, moves, copying, borrowing, lifetimes, destruction, nullability, bounds, patterns, generics, Result , collections, strings, concurrency, unsafe boundaries, runtime behavior, FFI, ABI, diagnostics, and regressions.

A negative test passes when the implementation rejects a program that violates a normative rule. Compilation failure due solely to an implementation limit or resource exhaustion does not by itself establish successful conformance testing.

A positive test passes when the implementation accepts a program conforming to the applicable edition and provides behavior consistent with the specification.

82. Compatibility

A CobaltC 1.0.1 program has stable meaning under conforming implementations, subject only to choices explicitly identified as implementation-defined or unspecified by this specification.

Optimization MUST NOT change specified observable semantics.

Binary compatibility is separate from source compatibility and depends on the applicable ABI profile.

83. Versioning

CobaltC 1.0.1 is a closed language edition. Corrections may clarify or repair the published specification, but a semantic change MUST NOT silently be presented as established CobaltC 1.0.1 behavior.

New keywords, declaration forms, expression forms, type categories, ownership mechanisms, borrowing mechanisms, lifetime syntax, concurrency primitives, module syntax, visibility syntax, macro facilities, or dynamic-dispatch facilities belong in a subsequent language edition.

Cross-Reference Integrity Clarifications

The following rules define interactions between language subsystems. They do not introduce additional features; they clarify that subsystem guarantees compose according to their existing definitions.

Ownership, borrowing, and lifetime rules apply across ordinary functions, associated functions, generic instantiations, collections, and foreign-function interfaces unless a specific unsafe boundary applies.

Result<T,E> represents an ordinary program value. It does not transfer ownership, bypass borrowing rules, or alter destruction semantics unless the contained types and operations explicitly do so.

defer , destruction hooks, and scope cleanup MUST preserve the ownership rules of the values they operate on. Cleanup mechanisms do not create additional access capabilities.

An unsafe context permits unsafe operations but does not disable safe-language rules for values and operations that remain within the safe model.

Foreign calls are treated as explicit safety boundaries. Any ownership transfer, lifetime relationship, aliasing guarantee, or representation requirement crossing that boundary MUST be established by the foreign interface declaration or contract.

Concurrency facilities provide synchronization guarantees only where their own semantics specify them. Ownership prevents invalid memory access and data races but does not define higher-level program coordination.

84. Final Safety Theorem

A conforming implementation executing conforming safe CobaltC code MUST NOT permit execution that violates the ownership, initialization, borrowing, lifetime, nullability, bounds, or synchronization requirements defined by this specification.

In particular, safe CobaltC cannot be used to create:

use-before-initialization;

use-after-move;

double ownership;

invalid borrow lifetime;

conflicting mutable aliasing;

unchecked nullable dereference;

unchecked safe out-of-bounds access;

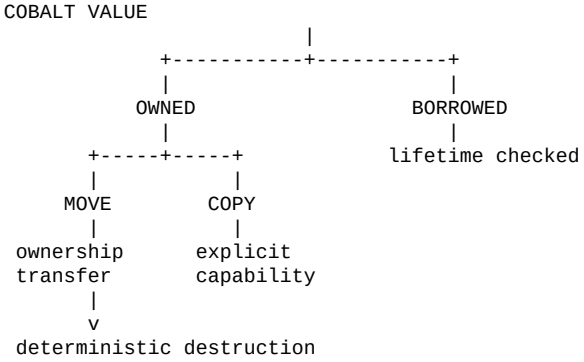
ordinary unsynchronized data races.

Unsafe and foreign code lie outside these automatic guarantees. A safe abstraction over unsafe code remains safe only while it maintains the invariants promised by its interface.

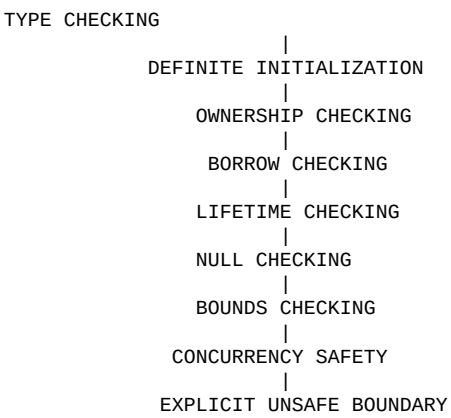
An implementation MAY reject a program conservatively where this specification permits conservative analysis, but it MUST NOT accept a program whose execution would violate a normative safety requirement.

85. Final Reference Model

The complete language model is:



with the following static safety layers:



86. Final Status

CobaltC Programming Language Specification 1.0.1

Status: Final Specification

The language design for CobaltC 1.0.1 is established by this document, which serves as the consolidated normative baseline for the edition. Editorial corrections may be made before final publication; new language features or changes to established language semantics require a subsequent language edition.

87. Illustrative Program

This section provides a small, non-normative CobaltC program illustrating the general syntax and style of the language. The program is intentionally simple and does not attempt to demonstrate a complete application or depend on facilities beyond those already described by this specification.

The example demonstrates module declaration, selective imports, constants, enumerations, structs, associated functions, mutable bindings, borrowing, assignment, conditional expression syntax, loops, function calls, and the main entry point.

```
module example;

import io { print, println };
```

```
const i32 limit = 3;

enum State
{
    Ready,
    Running,
    Finished
}

struct Counter
{
    i32 value;
}

fn Counter::increment(mut Counter* counter)
{
    counter->value = counter->value + 1;
}

fn describe(State state) : String
{
    return match state
    {
        Ready    => "ready",
        Running  => "running",
        Finished => "finished"
    };
}

fn main()
{
    mut Counter counter = Counter
    {
        value = 0
    };

    println(describe(Ready)); // ready

    while (counter.value < limit)
    {
        // Temporary mutable borrow of counter for the duration of the call.
        Counter::increment(&mut counter);

        print("count = ");
        println(counter.value);
    }

    println(describe(Finished)); // finished
}
```

Expected output: ready count = 1 count = 2 count = 3 finished The example is intended to show the syntactic character of CobaltC rather than prescribe a particular programming style. In particular, the explicit parameter to Counter::increment demonstrates that associated functions do not acquire an implicit receiver. The expression &mut counter explicitly supplies the mutable borrow required by the function parameter. The example also illustrates that owned values and borrowed access are distinct. counter remains the owner of its value while the mutable managed pointer is used to provide temporary exclusive access to it. The match expression demonstrates expression-oriented selection, while the while statement demonstrates ordinary iterative control flow. The program deliberately avoids introducing language features that are not otherwise necessary to demonstrate the basic syntax. ← Previous Bibliography ->

```
ready
count = 1
count = 2
count = 3
finished
```

The example is intended to show the syntactic character of CobaltC rather than prescribe a particular programming style. In particular, the explicit parameter to Counter::increment demonstrates that associated functions do not acquire an implicit receiver. The expression &mut counter explicitly supplies the mutable borrow required by the function parameter.

The example also illustrates that owned values and borrowed access are distinct. counter remains the owner of its value while the mutable managed pointer is used to provide temporary exclusive access to it.

The match expression demonstrates expression-oriented selection, while the while statement demonstrates ordinary iterative control flow. The program deliberately avoids introducing language features that are not otherwise necessary to demonstrate the basic syntax.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License .